

# Scraftie Architecture Docs

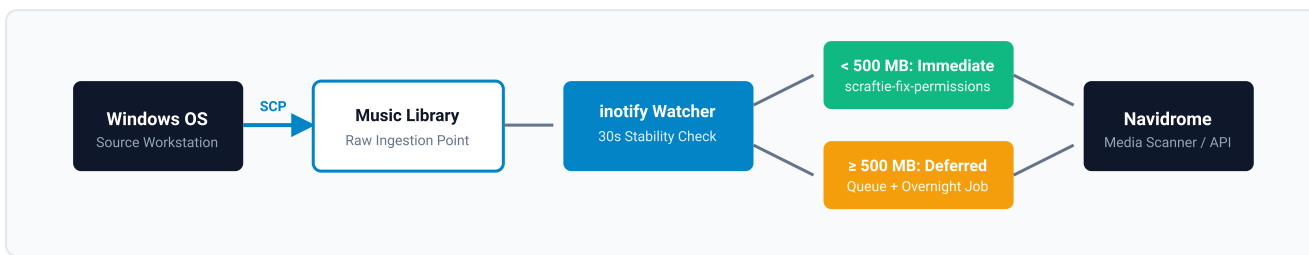
## AUTOMATIC MUSIC LIBRARY PERMISSION MANAGEMENT & AUTOMATION

System Engineering Report

### 1. EXECUTIVE SUMMARY & ARCHITECTURE CONTEXT

**Scraftie** is a self-hosted personal music streaming server running Linux with Navidrome as its core media engine. Music files are transferred from a local Windows workstation to the server over Secure Copy Protocol (SCP) directly into a dedicated media repository.

Initially, cross-platform file uploads suffered from permissions friction: directories uploaded via SCP received restrictive ACL masks, preventing the Navidrome service user from traversing directory trees and indexing new albums. This document records the technical evolution, empirical findings, and deployment of a resilient, event-driven server-side permission automation suite.



### 2. PROBLEM STATEMENT & ROOT CAUSE ANALYSIS

The initial music repository was owned exclusively by the system service user running Navidrome (e.g., `navidrome:navidrome`). This introduced two distinct permission failures during routine library maintenance and ingestion:

- Upload Access Denied:** Administrative SSH/SCP users could not edit, rename, or write into existing directory structures managed by the Navidrome service account.
- Directory Traversal Blocker:** Directories generated remotely via Windows SCP arrived on the Linux file system with restrictive mode masks. While individual media files inside held readable modes (e.g., `0644`), parent folders lacked group execute/search bits (`+x`), halting Navidrome's recursive scanner.

#### Core Distinction: Directory vs. File Access Modes

In POSIX file systems, a readable file (`-rw-r--r--`) is inaccessible if any parent directory lacks execute permissions (`d--r-x---`). Execute flags on directories dictate *traversal capability*. Windows SCP client session negotiation defaulted to masks that stripped group execute bits on created folders.

### 3. SHARED SECURITY GROUP & BASE PERMISSION ARCHITECTURE

To establish secure multi-user collaboration without over-granting access to unprivileged system accounts, a dedicated Linux system group (e.g., `music-managers`) was established. Both the primary administrative upload user and the `navidrome` daemon account were assigned as members.

#### Group Ownership & Setgid Bit

Applying the setgid bit (`chmod g+s`) to library directories ensures that all subdirectories and files created inside automatically inherit the parent group ownership (`music-managers`), rather than the uploader's primary default group.

#### Default Access Control Lists (ACLs)

Standard POSIX umask inheritance is often overridden by remote SCP client configurations. POSIX Default ACLs were applied to the root library directory to mandate default read, write, and traverse permissions for the shared group on all future child objects.

## 4. THE ACL MASK ANOMALY & NORMALIZATION REQUIREMENT

During integration testing, an empirical discrepancy was discovered between native Linux directory creation and remote Windows SCP ingestion:

| Ingestion Method                       | Created Object Type | Effective Mode / ACL Structure                                       | Navidrome Access Status         |
|----------------------------------------|---------------------|----------------------------------------------------------------------|---------------------------------|
| Native Linux<br>( <code>mkdir</code> ) | Directory           | <code>drwxrwsr-x+</code> (Inherits default ACL & setgid)             | <b>SUCCESS</b> (Full Traversal) |
| Windows SCP / SFTP                     | Directory           | <code>drwxr-xr-x#</code> (Effective mask restricts group write/exec) | <b>FAILED</b> (Access Denied)   |

### Empirical Discovery

Because SCP clients negotiate their own file creation attributes and permission masks, passive inherited Default ACLs alone are insufficient to guarantee access. An explicit permission normalization step was required to enforce mode integrity post-ingestion.

## 5. PERMISSION NORMALIZATION ENGINE

To reliably standardize the file tree, a specialized normalization utility was engineered: `/usr/local/bin/scraftie-fix-permissions`. The script validates path boundaries, applies required POSIX modes, recalculates ACL masks, and restores setgid bits recursively.

```
#!/usr/bin/env bash # /usr/local/bin/scraftie-fix-permissions - Target Normalization Utility
set -euo pipefail TARGET_DIR="${1:-}" MUSIC_GROUP="music-managers" if [[ -z "$TARGET_DIR" || ! -d "$TARGET_DIR" ]]; then echo "Error: Valid target directory required." >&2 exit 1 fi # 1. Enforce ownership and Setgid directory bits chown -R :"$MUSIC_GROUP" "$TARGET_DIR" find "$TARGET_DIR" -type d -exec chmod 2775 {} + # 2. Enforce standard file read/write permissions find "$TARGET_DIR" -type f -exec chmod 0664 {} + # 3. Apply Default and Explicit POSIX ACLs for group write/traverse access setfacl -R -m g::rwx,d:g::rwx "$TARGET_DIR" setfacl -R -m m::rwx "$TARGET_DIR" # Reset restrictive ACL masks
```

## 6. FILE SYSTEM EVENT MONITORING (`INOTIFY`)

Requiring manual terminal execution after every album upload defeated the goal of a seamless media server. To achieve hands-off operation, a continuous filesystem event watcher was developed using `inotifywait`: `/usr/local/bin/scraftie-permission-watcher`.

The watcher targets directory creation and movement events (`CREATE, ISDIR` and `MOVED_TO, ISDIR`). It ignores isolated file modification events to avoid redundant processing while files are actively streaming over network interfaces.

## 7. UPLOAD STABILITY & RACE-CONDITION GUARD

A common failure mode in filesystem monitoring is reacting to a directory creation event while remote file writes are still actively in progress. Triggering permission changes mid-transfer can corrupt transfer handles or process incomplete folder structures.

### 30-Second Size-Polling Algorithm

When a new directory event is detected, the watcher enters an inspection loop:

- 1. Measure total directory footprint using `du -s --block-size=1`.
- 2. Sleep for a designated stability interval (**30 seconds**).

## 8. TIERED PROCESSING ARCHITECTURE & DEFERRED QUEUE

Executing recursive file system operations across multi-gigabyte library additions can generate high I/O wait times, starving Navidrome's database engine of disk bandwidth. To prevent performance degradation, a tiered threshold model was implemented:

### Small Albums (< 500 MB)

Processed **immediately** upon passing the stability check. The watcher invokes `scraftie-fix-permissions` directly. Navidrome indexes the content within seconds.

### Large Ingestions (≥ 500 MB)

Path is appended to a persistent queue file:  
`/var/lib/scraftie/pending-permissions.txt`.  
Heavy I/O is deferred to low-traffic off-peak maintenance windows.

### Queue Integrity & Deduplication

The watcher checks existing queue entries before writing. If a target path already resides in `pending-permissions.txt`, duplicate insertions are suppressed to prevent redundant batch execution cycles during off-peak processing.

## 9. OVERNIGHT BATCH PROCESSOR

Deferred jobs are handled by a dedicated processing script: `/usr/local/bin/scraftie-overnight-fix`. This component acts as a batch processor and fault-tolerant retry engine.

- **Path Validation:** Reads lines from the queue and verifies that each entry exists and strictly resides within the authorized music library directory tree (preventing path traversal).
- **Execution & Atomic Clean:** Runs `scraftie-fix-permissions` against each valid directory. Successful operations are purged from the queue file atomically.
- **Failure Retention:** If an operation fails (e.g., target directory temporarily locked or unmounted), the item is retained in the queue file for retry during the next scheduled cycle.

## 10. SYSTEMD SERVICE & SCHEDULE INTEGRATION

To guarantee persistent execution across server reboots, both components were integrated into `systemd` via a background service daemon and an automated timer unit.

```
# /etc/systemd/system/scraftie-permission-watcher.service [Unit] Description=Scraftie Music Library Inotify Permission Watcher After=network.target local-fs.target [Service] Type=simple ExecStart=/usr/local/bin/scraftie-permission-watcher Restart=always RestartSec=10 User=root [Install] WantedBy=multi-user.target
```

```
# /etc/systemd/system/scraftie-overnight-fix.timer [Unit] Description=Run Scraftie Permission Maintenance Daily at 1:00 AM [Timer] OnCalendar=*-*-* 01:00:00 Persistent=true [Install] WantedBy=timers.target
```

## 11. TESTING & VERIFICATION MATRIX

To ensure production stability, every stage of the permission management pipeline underwent empirical validation on the live Linux host environment.

| ID | Test Scenario                   | Procedure / Test Input                                                               | Observed Result                                                                                   | Status   |
|----|---------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------|
| T1 | Manual Permission Fix           | Execute <code>scraftie-fix-permissions</code> against an unreadable album directory. | Permissions updated to <code>2775 / 0664</code> . Navidrome indexed album immediately.            | PASSED   |
| T2 | Direct Linux Directory Creation | Run <code>mkdir</code> inside the music root as administrative Linux user.           | Directory inherited group ownership and default ACL structure correctly.                          | PASSED   |
| T3 | Windows SCP Transfer            | Transfer test folder from Windows client via SCP.                                    | Reproduced restrictive mask anomaly (group traverse lost), proving need for watcher.              | VERIFIED |
| T4 | Automatic Small Upload (<500MB) | Upload a ~45 MB album folder via Windows SCP while watcher service active.           | Watcher detected folder, waited 30s stability period, ran fix. Navidrome indexed album.           | PASSED   |
| T5 | Automatic Queue Handling        | Temporarily set threshold to 1 MB; upload 5 MB file via SCP.                         | Directory skipped immediate processing and was written to <code>pending-permissions.txt</code> .  | PASSED   |
| T6 | Overnight Processor Manual Run  | Execute <code>scraftie-overnight-fix</code> with items in queue.                     | Batch fixed permissions on queued target, removed entry from file upon completion.                | PASSED   |
| T7 | Systemd Timer Verification      | Inspect systemd timers via <code>systemctl list-timers</code> .                      | <code>scraftie-overnight-fix.timer</code> confirmed active and scheduled for next 01:00 AM cycle. | PASSED   |

## 12. FINAL ARCHITECTURE OVERVIEW

The automated permission management suite converts raw multi-platform file ingestion into a structured, self-healing pipeline.

### System Capabilities Summary

- **Shared Group Security Model:** Grants explicit, least-privilege read/write/traverse rights to administrators and Navidrome without exposing files system-wide.
- **Active Normalization:** Corrects ACL mask degradation introduced by cross-platform Windows-to-Linux SCP file transfers.
- **Transfer Stability Guard:** Prevents premature file processing during active network transfers via size polling.
- **Resource-Aware Tiering:** Processes lightweight content immediately while deferring heavy batch jobs to off-peak hours.
- **Resilient Systemd Integration:** Provides daemon auto-restart, persistent timers, and queue-based failure recovery.

## 13. KEY ENGINEERING LESSONS

### Beyond Basic Permissions

Resolving cross-platform file sharing friction requires looking beyond basic POSIX modes ( `chmod 777` ). The ultimate solution combined Linux group ownership, POSIX ACLs, real-time filesystem event monitoring, stability polling, and background job scheduling to build a resilient, automated media pipeline.

